

CS2 AI COACHING SYSTEM

Ultimate CS2 Coach

Parte 1B — Os Sentidos e o Especialista

RAP Coach Model, ChronovisorScanner, GhostEngine, Fontes de Dados,
HLTV, Steam, FACEIT

Autor: Renan Augusto Macena

Agosto 2026

Ultimate CS2 Coach — Parte 1B: Os Sentidos e o Especialista

Topicos: Modelo RAP Coach (arquitetura de 7 componentes: Percepcao, Memoria LTC+Hopfield, Estrategia, Pedagogia, Atribuicao Causal, Posicionamento, Comunicacao), ChronovisorScanner (deteccao multi-escala de momentos criticos), GhostEngine (pipeline de inferencia 4-tensores), e todas as fontes de dados externas (Demo Parser, HLTV, Steam, FACEIT, TensorFactory, FAISS, Round Context).

Autor: Renan Augusto Macena

Revisao base: em sincronia com [Book-Coach-1B.md](#) (canonico italiano) em 2026-08-15.

Este documento e a continuacao da **Parte 1A** — *O Cerebro: Arquitetura Neural e Treinamento*, que documenta os modelos de rede neural (JEPA, VL-JEPA, AdvancedCoachNN), o sistema de treinamento de 3 niveis de maturidade, o Observatorio de Introspeccao do Coach, e os fundamentos arquiteturais do sistema (principio NO-WALLHACK, contrato 25-dim).

Indice

Parte 1B — Os Sentidos e o Especialista (este documento)

4. Subsistema 2 — Modelo RAP Coach ([backend/nn/experimental/rap_coach/](#))

- RAPCoachModel (Entradas Visuais Duais: por-timestep e estaticas)
- Camada de Percepcao (ResNet de 3 fluxos)
- Camada de Memoria (LTC + Hopfield)
- Camada de Estrategia (SuperpositionLayer + MoE)
- Camada Pedagogica (Value Critic + Atribuicao Causal)
- Modelo Latente de Habilidades
- RAP Trainer (Funcao de Perda Composta)
- ChronovisorScanner (Deteccao Multi-Escala de Momentos Criticos)
- GhostEngine (Pipeline de Inferencia 4-Tensores com PlayerKnowledge)

5. Subsistema 1B — Fontes de Dados ([backend/data_sources/](#))

- Demo Parser + Demo Format Adapter
- Event Registry (Schema de Eventos CS2)

- Trade Kill Detector
- Steam API + Steam Demo Finder
- Modulo HLTV (stat_fetcher, FlareSolvrr, Docker, rate limiting adaptativo)
- FACEIT API + Integration
- TensorFactory — Fabrica de Tensores (Percepcao Player-POV NO-WALLHACK)
- Indice Vetorial FAISS (Busca Semantica de Alta Velocidade)
- Contexto dos Rounds (Grade Temporal)

Parte 1A — O Cerebro: Nucleo da Rede Neural (JEPA, VL-JEPA, AdvancedCoachNN, SuperpositionLayer, EMA, CoachTrainingManager, TrainingOrchestrator, ModelFactory, NeuralRoleHead, MaturityObservatory)

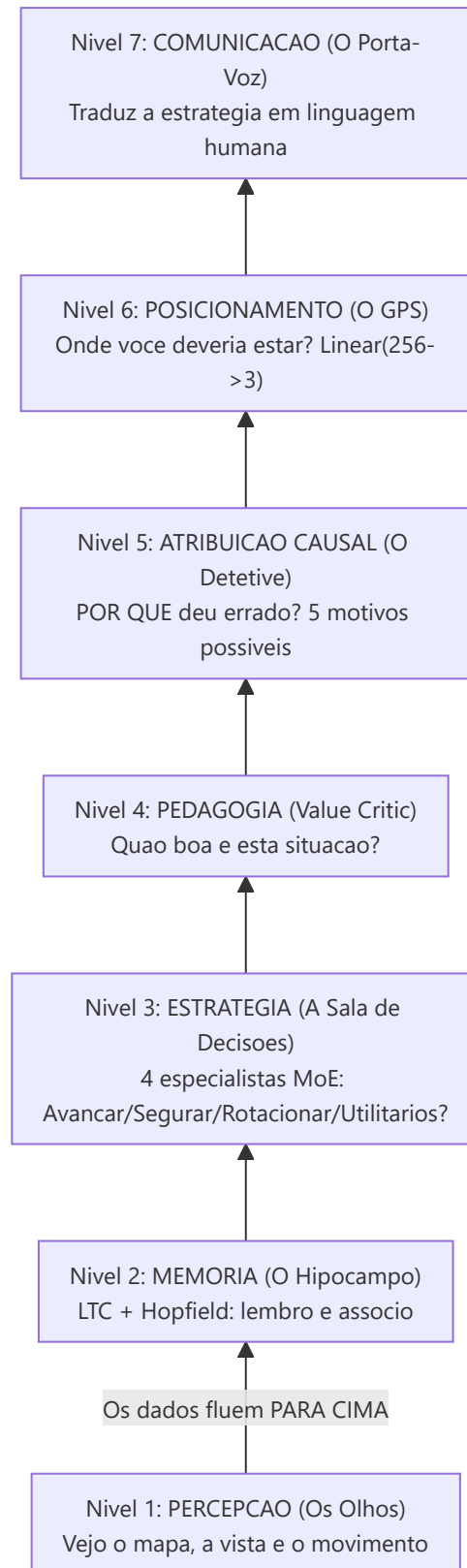
Parte 2 — Secoes 5-13: Servicos de Coaching, Coaching Engines, Conhecimento e Recuperacao, Motores de Analise (11), Processamento e Feature Engineering, Modulo de Controle, Progresso e Tendencias, Banco de Dados e Storage (Tri-Tier), Pipeline de Treinamento e Orquestracao, Funcoes de Perda

Parte 3 — Logica do Programa, UI, Ingestao, Tools, Tests, Build, Remediation

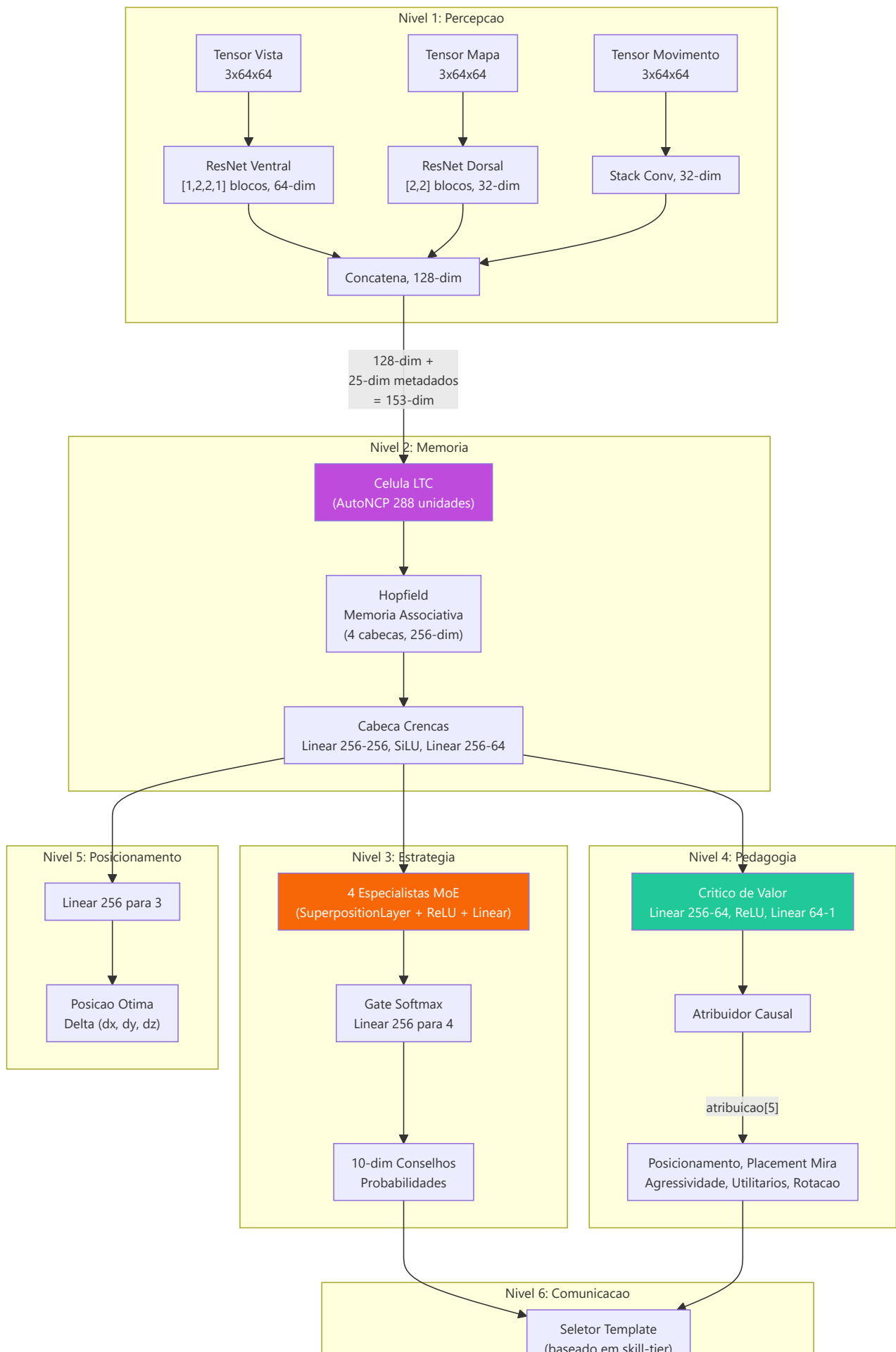
4. Subsistema 2 — Modelo RAP Coach

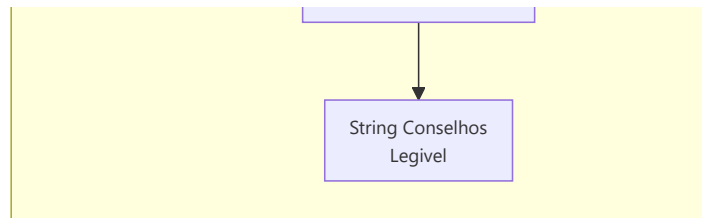
Diretorio canonico: `backend/nn/experimental/rap_coach/` (o caminho antigo `backend/nn/rap_coach/` e um shim de redirecionamento) **Arquivos:** `model.py`, `perception.py`, `memory.py`, `strategy.py`, `pedagogy.py`, `communication.py`, `skill_model.py`, `trainer.py`, `chronovisor_scanner.py`

O RAP (Reasoning, Adaptation, Pedagogy) Coach e uma **arquitetura profunda com 6 componentes neurais aprendiveis + 1 camada de comunicacao externa**, especificamente projetada para coaching CS2 em condicoes de observabilidade parcial (condicoes POMDP). A classe `RAPCoachModel` contem Percepcao (`RAPPerception`), Memoria (`RAPMemory` com LTC+Hopfield), Estrategia (`RAPStrategy`), Pedagogia (`RAPPedagogy` com Value Critic e Skill Adapter), Atribuicao Causal (`CausalAttributor`) e uma Cabeça de Posicionamento (`nn.Linear(256→3)`), todos aprendiveis. A camada de Comunicacao (`communication.py`) opera externamente como seletor de templates de pos-processamento. O forward pass produz 6 saidas: `advice_probs`, `belief_state`, `value_estimate`, `gate_weights`, `optimal_pos` e `attribution`.







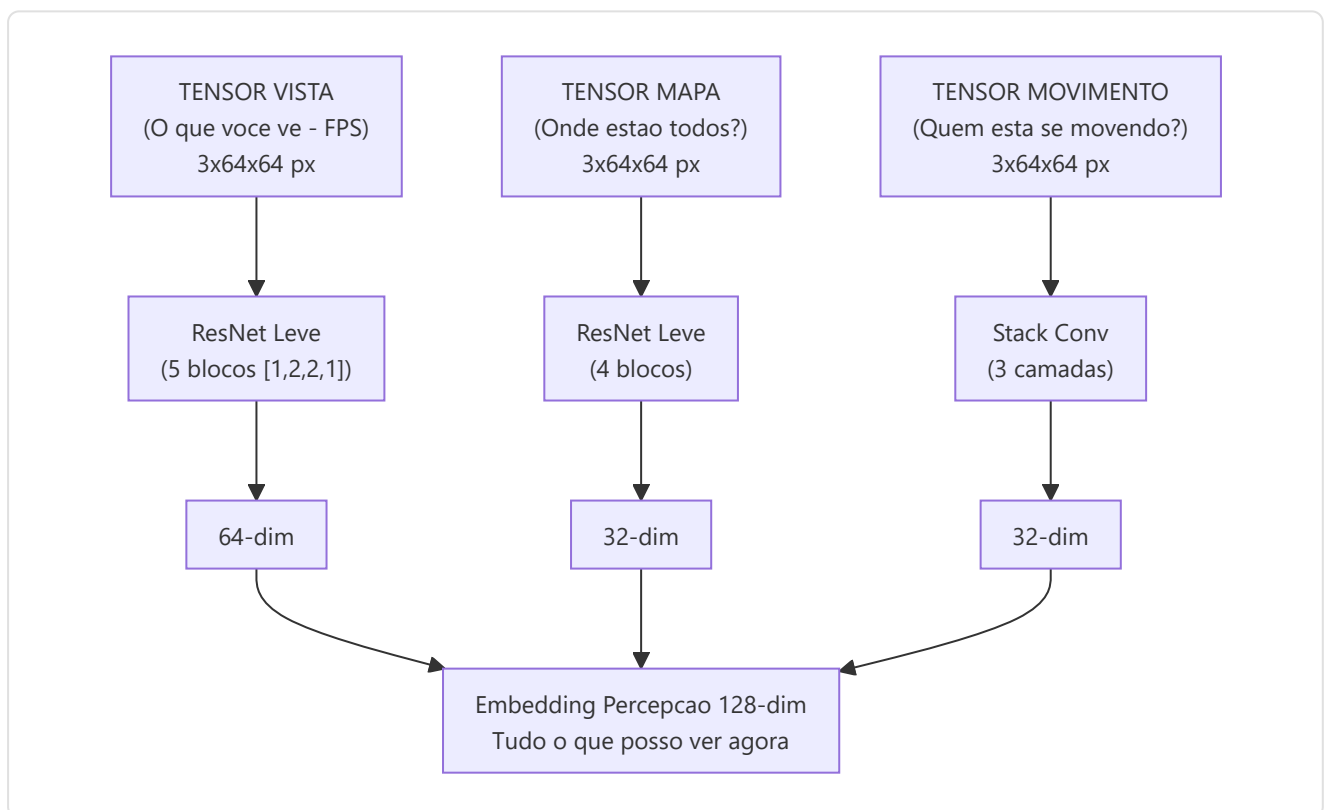


-Camada de percepcao (`perception.py`)

Um front-end **convolucional de tres fluxos** que processa as entradas visuais:

Entrada	Forma	Backbone	Dim Saida
Tensor de visualizacao	3x64x64	Fluxo ventral ResNet: [1,2,2,1] blocos, 3->64 canais	64-dim
Tensor de mapa	3x64x64	Fluxo dorsal ResNet: [2,2] blocos, 3->32 canais	32-dim
Tensor de movimento	3x64x64	Conv(3->16->32) + MaxPool + AdaptiveAvgPool	32-dim

Os tres vetores de caracteristicas sao concatenados em um unico **embedding de percepcao de 128 dimensoes** (64 + 32 + 32).



Os blocos ResNet usam **atalhos de identidade** com downsample aprendivel (Conv1x1 + BatchNorm) quando stride != 1 ou o numero de canais muda. **24 camadas de convolucao** em todos os tres fluxos:

Fluxo	Configuracao bloco	Blocos	Conv/Bloco	Conv atalhos	Total
Vista (Ventral)	<code>[1,2,2,1]</code> -> 1 + 5 = 6 blocos	6	2	1 (primeiro bloco)	13
Mapa (Dorsal)	<code>[2,2]</code> -> 1 + 3 = 4 blocos	4	2	1 (primeiro bloco)	9
Movimento	Stack de conversao (2 camadas)	—	—	—	2
Total					24

Como funciona `_make_resnet_stack` : Cria 1 bloco inicial com `stride=2` (para downsampling espacial), depois `sum(num_blocks) - 1` blocos adicionais com `stride=1` . Cada `ResNetBlock` tem 2 camadas Conv2d (kernel 3x3). O primeiro bloco tambem recebe um atalho Conv1x1 porque os canais de entrada (3) sao diferentes dos canais de saida (64 ou 32).

Nota sobre escolha arquitetonica (F3-29): A configuracao original `[3,4,6,3]` (15 blocos, 33 conv no fluxo ventral) foi projetada para entradas 224x224 (o tamanho padrao do ImageNet). Para entradas 64x64 como as usadas neste projeto, as feature maps colapsariam espacialmente apos o primeiro bloco stride-2, tornando os blocos subsequentes redundantes. A configuracao `[1,2,2,1]` (5 blocos efetivos) e calibrada especificamente para a resolucao de treinamento 64x64, com `AdaptiveAvgPool2d` lidando com qualquer resolucao espacial residual. Checkpoints anteriores sao automaticamente detectados como `_stale_checkpoint` por `load_nn()` .

-Camada de memoria (`memory.py`) — LTC + Hopfield

Esta parte enfrenta o desafio fundamental de que o coach CS2 e um **Processo de Decisao de Markov parcialmente observavel** (POMDP).

Rede de constante de tempo liquida (LTC) com cabeamento AutoNCP:

- Entrada: 153 dim (128 percepcao + 25 metadados)
- Unidades NCP: **512** (`hidden_dim * 2` = 256 x 2), repartidas pelo wiring AutoNCP em **154 neuronios motores / 143 command / 215 inter**; a saida motora (154-dim) e projetada para 256-dim atraves de `ltc_projection`
- Usa a biblioteca `ncps` com padroes de conectividade esparsos, similares aos do cerebro; um `_patched_ode_solver` interno corrige um bug de batch-broadcast do `ncps`
- Adapta a resolucao temporal ao ritmo do jogo (configuracoes lentas vs. tiroteios rapidos)
- Seeding deterministico (NN-MEM-02): numpy + torch RNG seedados em 42 durante a criacao do wiring AutoNCP, com restauracao do estado RNG original apos a inicializacao — garante portabilidade dos checkpoints entre diferentes execucoes

Memoria associativa de Hopfield:

- Entrada/Saida: 256-dim
- Cabecas: 4

- Usa `hflayers.HopfieldLayer` com **32 padroes-prototipo treinaveis** (`quantity=32` , $32 \times 256 = 8.192$ parametros) como **memoria enderecavel por conteudo** para a recuperacao dos rounds prototipo
- Os prototipos podem ser inicializados via **K-means (K=32)** sobre os dados reais atraves de `initialize_prototypes()` (Fase 5B)

Atraso de ativacao Hopfield (NN-MEM-01 + RAP-M-04):

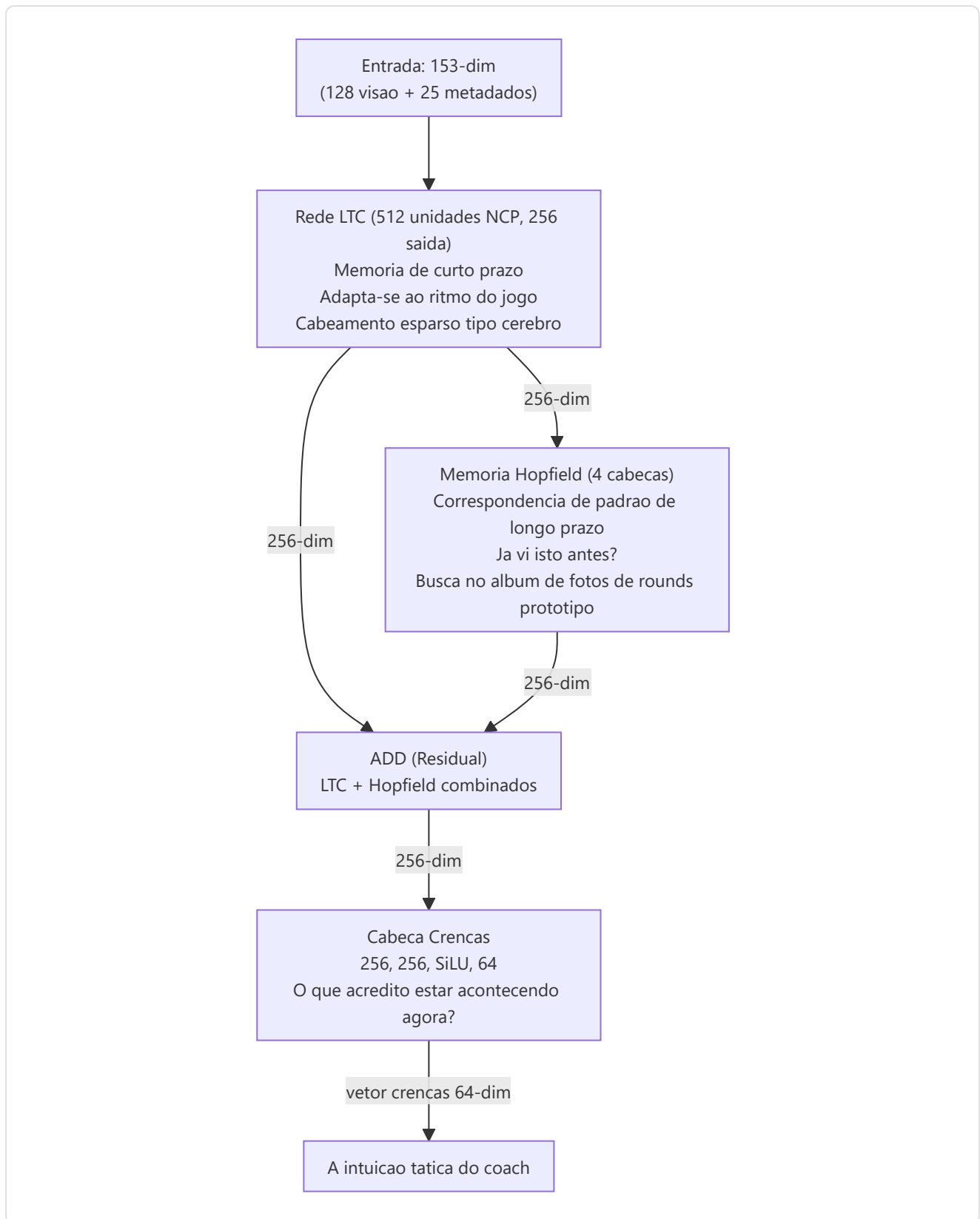
A rede de Hopfield **nao se ativa imediatamente** durante o treinamento: padroes ainda nao modelados produziram atencao quase uniforme em todos os slots, adicionando ruido em vez de sinal. Por isso:

- O flag `_hopfield_trained` permanece `False` ate que o trainer chame `notify_optimizer_step()` - ou seja, ate o **primeiro optimizer step real** que modelou os padroes
- Antes da ativacao, o forward pass retorna `torch.zeros_like(ltc_out)` no lugar da saida Hopfield
- O carregamento de um checkpoint (`load_state_dict`) define `_hopfield_trained = True` imediatamente, assumindo que o modelo ja foi treinado

RAPMemoryLite — Fallback LSTM puro:

Modulo substituto leve para `RAPMemory` , usado quando as dependencias `ncps` / `hflayers` nao estao disponiveis ou quando se deseja um modelo mais portavel:

- LSTM padrao PyTorch: `nn.LSTM(153, 256, batch_first=True)`
- Mesmo contrato I/O: Entrada `[B, T, 153]` -> Saida `(combined_state [B, T, 256], belief [B, T, 64], hidden)`
- Mesma cabeca de crenças: `Linear(256→256) → SiLU → Linear(256→64)`
- Nenhum seeding RNG necessario (sem AutoNCP)
- Nenhum atraso de treinamento Hopfield (sem padroes memorizados)
- Instanciado via `ModelFactory.TYPE_RAP_LITE` ("rap-lite") com `use_lite_memory=True`



Combinacao residual: `combined_state = ltc_out + hopfield_out`

Cabeca de conviccao: `Linear(256→256) → SiLU → Linear(256→64)` — produz um vetor de conviccao de 64 dimensoes que codifica a compreensao tatica latente do treinador.

Passo forward:

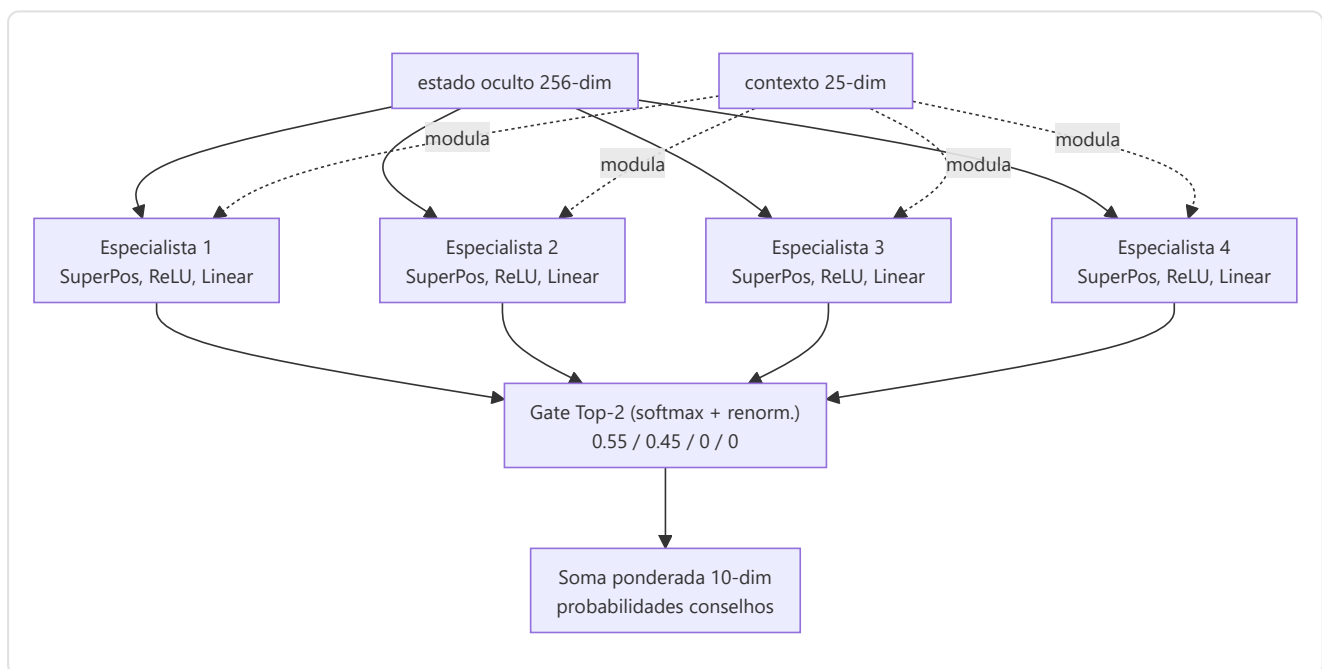
```
ltc_out, hidden = self.ltc(x, hidden) # x: [B, seq, 153] → [B, seq, 256]
mem_out = self.hopfield(ltc_out) # [B, seq, 256]
combined_state = ltc_out + mem_out # Residuo
belief = self.belief_head(combined_state) # [B, seq, 64]
return combined_state, belief, hidden
```

-Camada Estrategia (`strategy.py`) — Superposicao + MoE

Implementa **SuperpositionLayer** combinado com uma mistura de especialistas contextualizados com **roteamento esparso Top-2**:

SuperpositionLayer (`layers/superposition.py`): condicionamento **FiLM** onde `output = gamma(context) . (W.x + b) + beta(context)`, com `gamma = sigmoid(context_gate(context))` e `beta = context_beta(context)` (inicializado em zero). O gate multiplicativo mascara seletivamente as saidas dos especialistas; o termo aditivo permite ao contexto injetar features. A perda de esparsidade L1 (`context_gate_l1_weight = 1e-4`) incentiva um gating esparso e interpretavel. Observavel: as estatisticas do gate (media, std, sparsidade, active_ratio) podem ser rastreadas.

Nota: `RAPStrategy.__init__` usa `context_dim=89` - o contexto e a concatenacao de **metadados (25) + vetor de crenças (64)**. A rede de gate e `nn.Linear(hidden_dim=256, num_experts=4)` seguida de softmax e renormalizacao Top-2.



4 Modulos Especialistas: Cada especialista e composto por `SuperpositionLayer` (`FiLM, context_dim=89`) $\rightarrow$ `ReLU` $\rightarrow$ `Linear( $\rightarrow 10$ )`.

Gate Network: `nn.Linear(256 $\rightarrow$ 4)` (logits brutos) $\rightarrow$ softmax $\rightarrow$ **Top-2 renormalizado** (os outros dois especialistas ficam com peso zero).

Saida: Distribuicao de probabilidade de conselhos de 10 dimensoes e vetor de pesos de gate de 4 dimensoes.

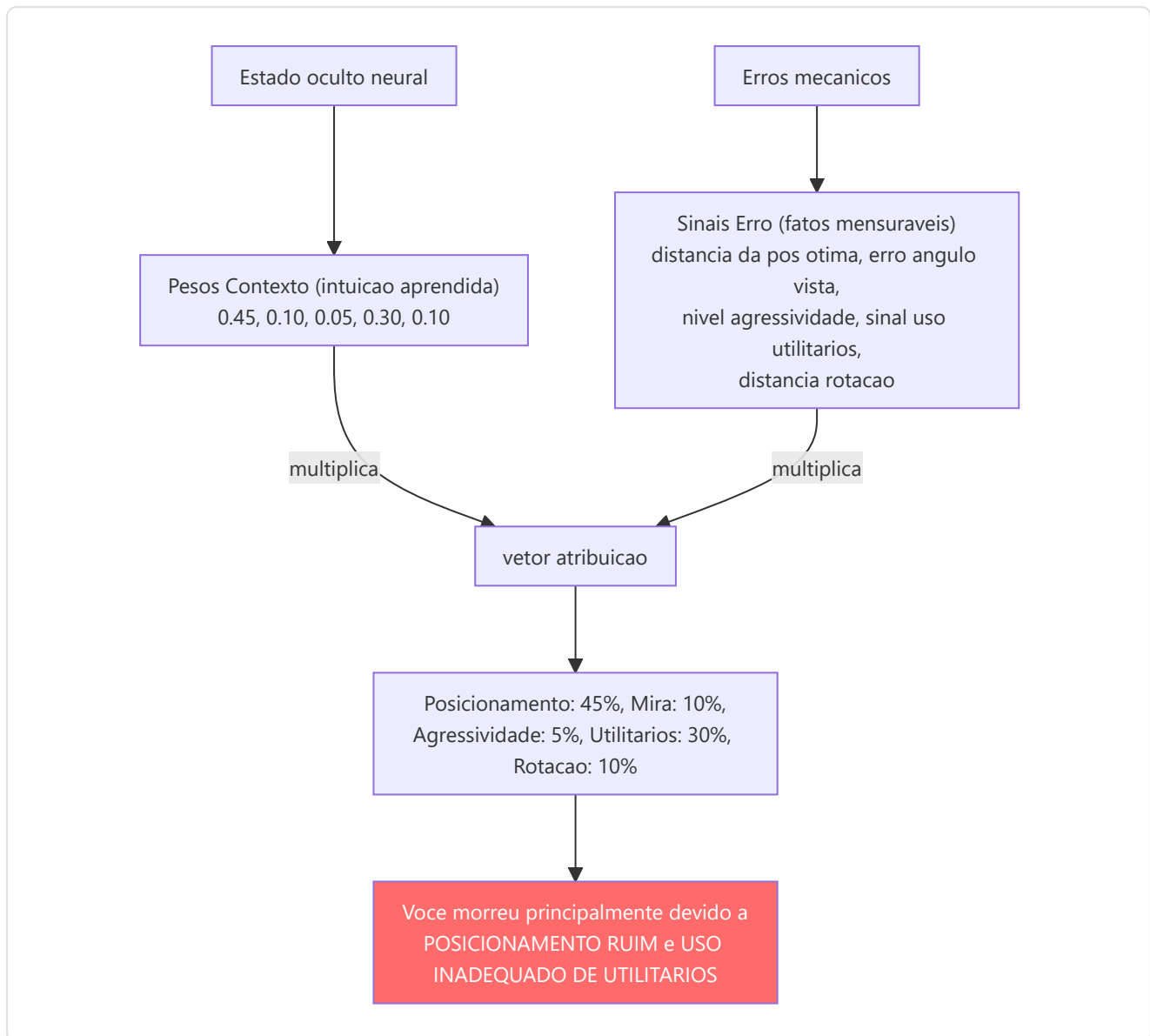
-Camada Pedagogica (`pedagogy.py`) — Valor + Atribuicao

Dois submodulos:

1. **Value Critic:** `Linear(256→64) → ReLU → Linear(64→1)` . Estima $V(s)$ para o aprendizado com diferencas temporais. **Skill Adapter:** `Linear(10 skill_buckets → 256)` permite estimativas de valor condicionadas pelas habilidades.
2. **CausalAttributor:** Produz um vetor de atribuicao de 5 dimensoes que mapeia os conceitos de treinamento:

Indice	Conceito	Sinal mecanico
0	Posicionamento	<code>norm(position_delta)</code>
1	Posicionamento da mira	<code>norm(view_delta)</code>
2	Agressao	<code>0,5 x position_delta</code>
3	Utilitarios	<code>sigmoid(hidden.mean())</code> — sinal aprendido e dependente do contexto : produz uma ativacao alta quando a rede detecta situacoes onde o uso de utilitarios era relevante, baixa quando o contexto tatico torna os utilitarios secundarios. Nao e um placeholder estatico, mas uma funcao nao-linear do estado oculto que se adapta durante o treinamento
4	Rotacao	<code>0,8 x position_delta</code>

Fusao: `atribuicao = context_weights x mechanical_errors` onde `context_weights` deriva de `Linear(256→32) → ReLU → Linear(32→5) → Sigmoid` .

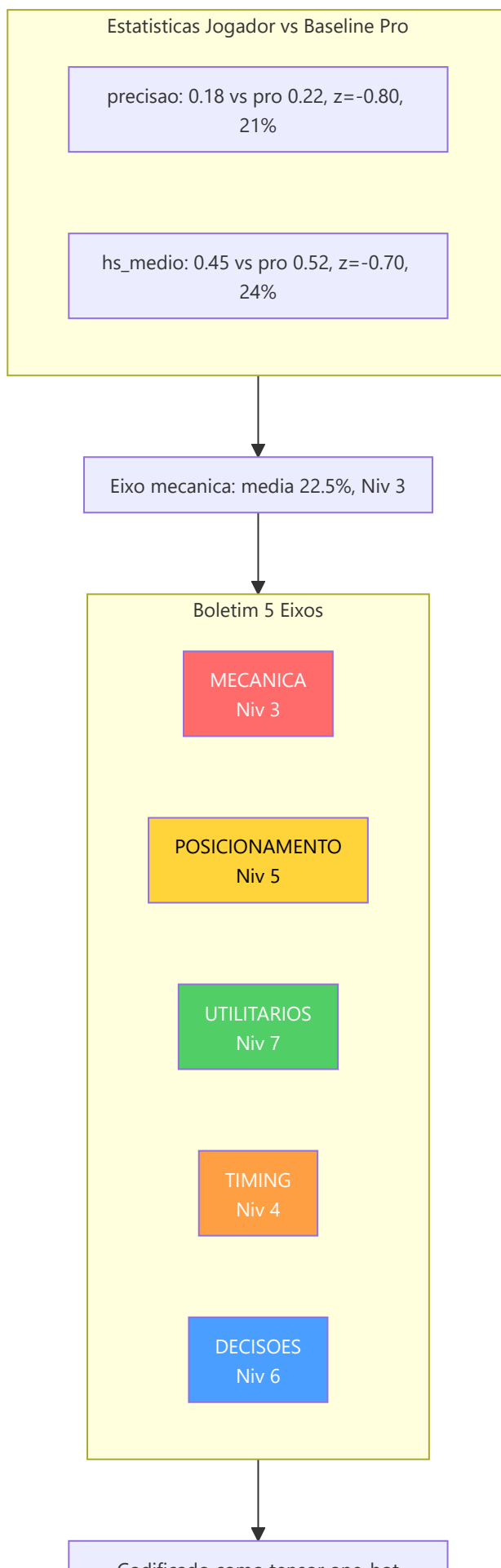


-Modelo latente das habilidades (`skill_model.py`)

Decompoe as estatísticas brutas em 5 eixos de habilidades usando a normalização estatística em relação às linhas de base dos profissionais:

Eixo de habilidades	Estatísticas de entrada	Normalização
Mecanicas	Precisao, avg_hs	Pontuacao Z (mu=pro_mean, sigma=pro_std)
Posicionamento	Avaliacao_sobrevivencia, avaliacao_kast	Pontuacao Z
Utilitarios	Utility_blind_time, Utility_inimigos_cegados	Pontuacao Z
Timing	Percentual_vitorias_duelo_abertura, Pontuacao_agressao_posicional	Pontuacao Z
Decisao	Percentual_vitorias_clutch, Impacto_avaliacao	Pontuacao Z





Alimentado ao Adaptador Skill da Camada Pedagogia

As pontuacoes Z sao convertidas em percentis via a **aproximacao logistica** $1/(1+\exp(-1,702z))$ (aproximacao CDF rapida), depois o percentil medio e mapeado para um **nivel curricular** (1-10) via `int(avg_skill * 9) + 1`, fixado em [1, 10]. O nivel e codificado como um tensor one-hot (10-dim) via `SkillLatentModel.get_skill_tensor()` para o adaptador de competencias da camada pedagogica.

-RAP Trainer (`trainer.py`)

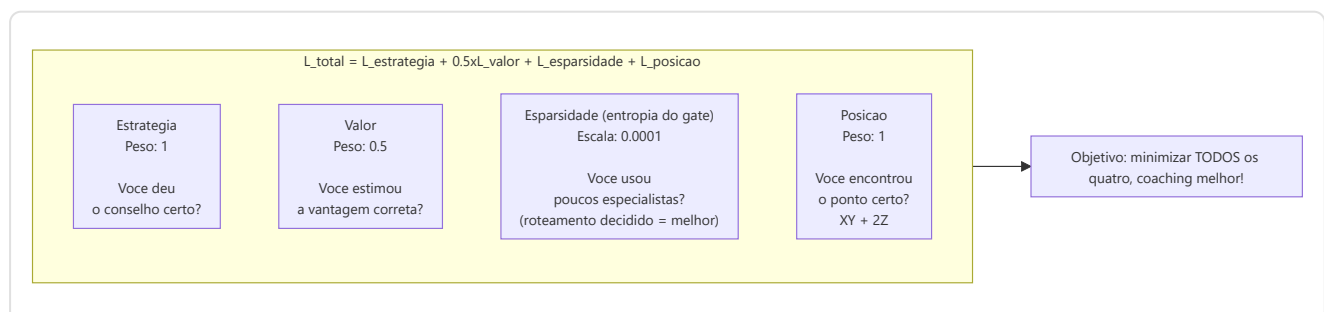
Orquestra o ciclo de treinamento com uma **funcao de perda composta**:

```
L_total = 1.0 x L_estrategia + 0.5 x L_valor + 1.0 x L_esparsidade + 1.0 x L_posicao
```

Termo de perda	Formula	Peso	Proposito
L_strategy	<code>MSELoss(advice_probs, target_strat)</code>	1.0	Recomendacao tatica correta
L_value	<code>MSELoss(V(s), true_advantage)</code> (com <code>val_mask</code> opcional)	0.5	Estimativa precisa da vantagem
L_sparsity	<code>model.compute_sparsity_loss(gate_weights)</code> — entropia do gate : <code>context_gate_l1_weight x (-sum p.log p)</code> , faixa [0, log 4 ~ 1.386], escala 1e-4 (parametro explicito, thread-safe)	1e-4	Especializacao de experts
L_position	<code>MSE(pred_x) + MSE(pred_y) + 2.0 x MSE(pred_z)</code>	1.0	Posicionamento otimo, penalidade rigorosa no eixo Z

Otimizacao: AdamW (lr=1e-4, weight_decay=1e-2), scheduler `CosineAnnealingLR(T_max=100, eta_min=1e-6)`, **acumulo de gradientes** em 4 passos (`accumulation_steps=4`), AMP `GradScaler` (somente CUDA).

Nota: O multiplicador 2x no eixo Z existe porque os erros de posicionamento vertical (por exemplo, um nível errado em Nuke/Vertigo) são taticamente catastróficos: representam erros de andar errado que nenhuma correção horizontal pode corrigir.



Saida por fase de treinamento: `{loss, sparsity_ratio, loss_pos, z_error}`.

-Resumo do passo forward do RAPCoachModel

Assinatura: `forward(view_frame, map_frame, motion_diff, metadata, skill_vec=None, hidden_state=None)` — o parametro `hidden_state` (NN-40) permite passar o estado recorrente da memoria de uma chamada a outra, habilitando **inferencia continua** sem cold-start: o GhostEngine pode manter a memoria entre ticks consecutivos em vez de reiniciar do zero a cada avaliacao.

Fix NN-39 — Entradas Visuais Duais: O passo forward gerencia dois formatos de entrada visual atraves de uma verificacao dimensional explicita:

Formato Entrada	Shape	Quando se usa	Comportamento
Por-timestep	<code>[B, T, C, H, W]</code> (5-dim)	Treinamento com sequencias temporais	Cada timestep processado individualmente pela CNN
Estatico	<code>[B, C, H, W]</code> (4-dim)	Inferencia em tempo real (GhostEngine)	Frame unico expandido sobre todos os timesteps

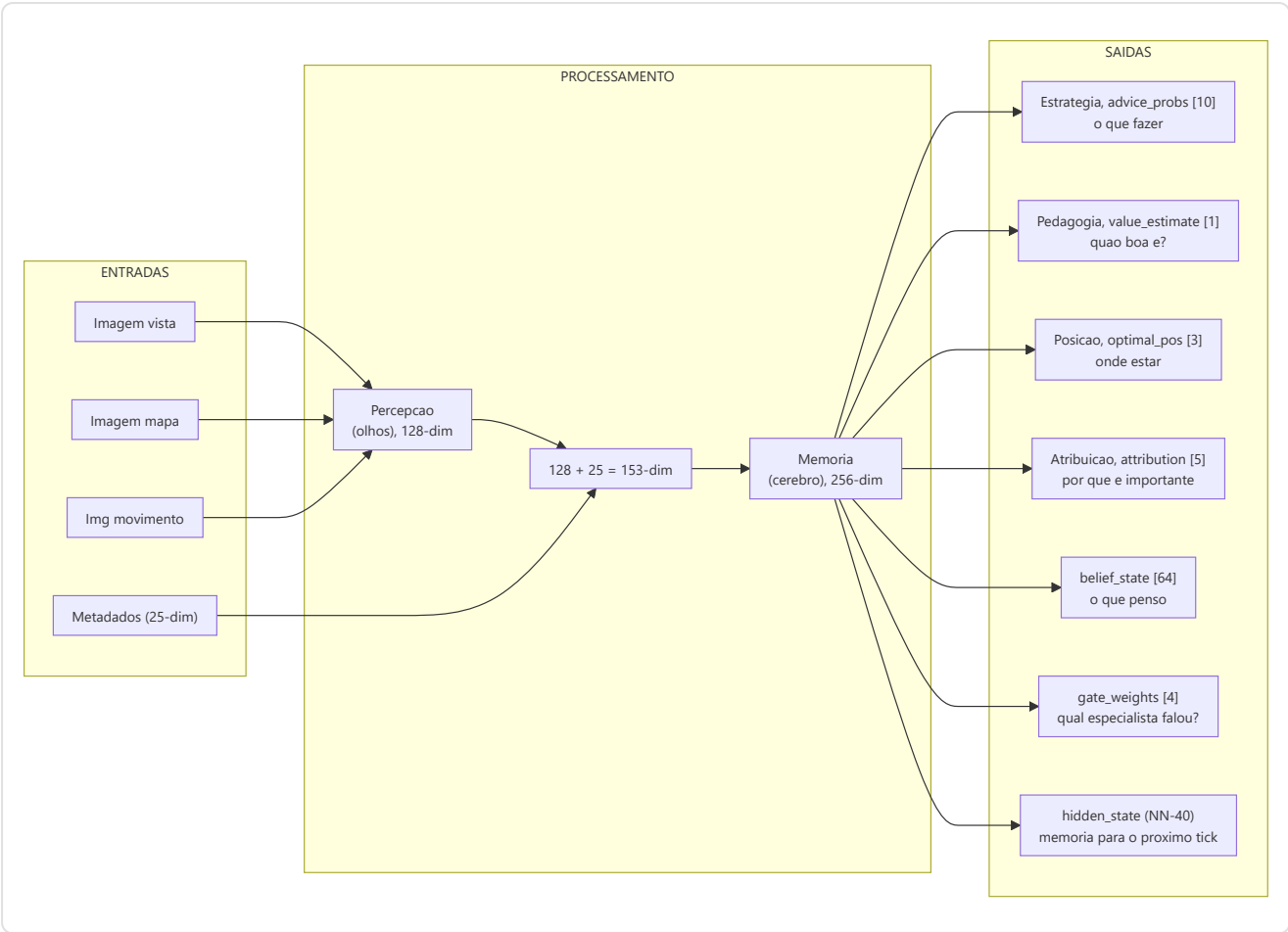
```

def forward(view_frame, map_frame, motion_diff, metadata, skill_vec=None):
    batch_size, seq_len, _ = metadata.shape

    # NN-39 fix: suporta entrada visual per-timestep [B,T,C,H,W] e estatica [B,C,H,W]
    if view_frame.dim() == 5:
        # Per-timestep - processa cada timestep atraves da CNN separadamente
        z_frames = []
        for t in range(view_frame.shape[1]):
            z_t = self.perception(view_frame[:, t], map_frame[:, t], motion_diff[:, t])
            z_frames.append(z_t)
        z_spatial_seq = torch.stack(z_frames, dim=1)      # [B, T, 128]
    else:
        # Estatico - frame unico expandido sobre todos os timesteps
        z_spatial = self.perception(view_frame, map_frame, motion_diff) # [B, 128]
        z_spatial_seq = z_spatial.unsqueeze(1).expand(-1, seq_len, -1) # [B, T, 128]

    lstm_in = cat([z_spatial_seq, metadata], dim=2)      # [B, seq, 153]
    hidden_seq, belief, new_hidden = self.memory(lstm_in, hidden=hidden_state) # [B, seq, 256],
    last_hidden = hidden_seq[:, -1, :]
    prediction, gate_weights = self.strategy(last_hidden, context) # [B, 10], [B, 4]
    value_v = self.pedagogy(last_hidden, skill_vec)      # [B, 1]
    optimal_pos = self.position_head(last_hidden)         # [B, 3]
    attribution = self.attributor.diagnose(last_hidden, optimal_pos) # [B, 5]
    return {
        "advice_probs": prediction,      # [B, 10]
        "belief_state": belief,           # [B, seq, 64]
        "value_estimate": value_v,        # [B, 1]
        "gate_weights": gate_weights,     # [B, 4]
        "optimal_pos": optimal_pos,       # [B, 3]
        "attribution": attribution,        # [B, 5]
        "hidden_state": new_hidden,       # NN-40: estado recorrente para inferencia continua
    }

```



-ChronovisorScanner (`chronovisor_scanner.py`)

Um **modulo de processamento de sinal multi-escala** que identifica os momentos criticos nas partidas analisando os deltas de vantagem temporal em **3 niveis de resolucao** (micro, standard, macro):

Configuracao Multi-Escala (`ANALYSIS_SCALES`):

Escala	Window (ticks)	Lag	Limiar	Descricao
Micro	64	16	0.10	Decisoes de engajamento sub-segundo
Standard	192	64	0.15	Momentos criticos a nivel de engajamento
Macro	640	128	0.20	Deteccao de mudancas estrategicas (5-10 segundos)

Pipeline de deteccao (para cada escala):

1. Usa o modelo RAP treinado para prever V(s) para cada tick window.
2. Calcula os deltas usando o lag configurado para a escala: `deltas = values[LAG:] - values[:-LAG]`.
3. Detecta os **picos** onde `|delta| > limiar` (variavel por escala: 0.10/0.15/0.20).
4. Busca o pico dentro da janela configurada, mantendo a coerencia do sinal.
5. A **supressao nao maxima** impede deteccoes duplicadas.
6. Classifica cada pico como **"jogada"** (gradiente positivo, vantagem adquirida) ou **"erro"** (negativo, vantagem perdida).

7. Retorna instancias da dataclass `CriticalMoment` com `(match_id, start_tick, peak_tick, end_tick, severity [0-1], type, description, scale)`.

Limite de segurança de tick (F3-21): `_MAX_TICKS_PER_SCAN = 50.000` — partidas com mais de 50K ticks (possivel com overtimes estendidos ou matches muito longos) sao **truncadas** com um warning (NN-CV-02) em vez de saturar a RAM. O sistema coleta `_MAX_TICKS_PER_SCAN + 1` ticks para detectar a truncagem e avisa que os momentos criticos da fase final da partida podem ser perdidos.

Deduplicacao cross-escala: Quando o mesmo momento e detectado em escalas diferentes (ex: um peek critico visivel tanto na escala micro quanto na standard), a deduplicacao prioriza **micro > standard > macro** (a escala mais fina vence). `MIN_GAP_TICKS = 64` (~1 segundo) define a distancia minima entre dois momentos: se dois picos estao mais proximos que 64 ticks, sao considerados o mesmo evento e apenas o de escala mais fina e mantido.

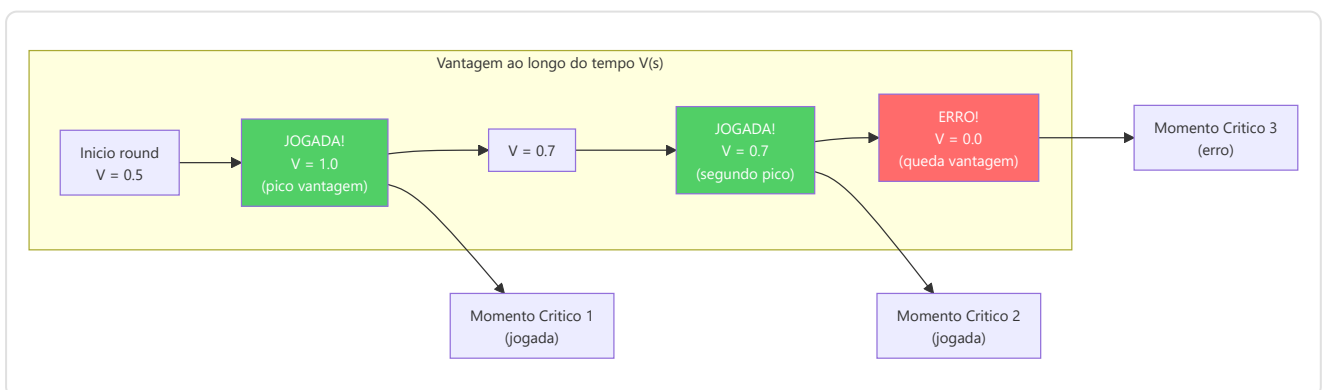
Rotulos de severidade: A severidade (0-1) e classificada automaticamente para o `MatchVisualizer`:

- `severity > 0.3` -> **"critical"** (momento que muda a partida)
- `severity > 0.15` -> **"significant"** (momento relevante)
- caso contrario -> **"notable"** (momento digno de nota)

ScanResult dataclass: Tipo de retorno estruturado que distingue sucesso de falha:

Campo	Tipo	Descricao
<code>critical_moments</code>	<code>List[CriticalMoment]</code>	Momentos criticos detectados
<code>success</code>	<code>bool</code>	True se o scan foi concluido (mesmo com 0 momentos)
<code>error_message</code>	<code>Optional[str]</code>	Detalhe de erro se <code>success=False</code>
<code>model_loaded</code>	<code>bool</code>	Se o modelo RAP estava disponivel
<code>ticks_analyzed</code>	<code>int</code>	Numero de ticks efetivamente analisados

Propriedades utilitarias: `is_empty_success` (scan bem-sucedido mas nenhum momento critico encontrado), `is_failure` (scan falhou — modelo nao carregado, erro de DB, etc.).



-GhostEngine (`inference/ghost_engine.py`)

Inferencia em tempo real para o "Ghost" — overlay da posicao otima do jogador. O GhostEngine representa o **ponto final** de toda a cadeia neural: e onde o RAP Coach Model produz saidas visiveis ao usuario sob a forma de um "jogador fantasma" no mapa tatico.

Pipeline de Inferencia 4-Tensores com PlayerKnowledge:

A pipeline de inferencia opera em 5 fases sequenciais para cada tick de reproducao:

Fase 1 — Carregamento do Modelo (`_load_brain()`)

- Verifica `USE_RAP_MODEL` da configuracao (interruptor geral)
- `ModelFactory.get_model(ModelFactory.TYPE_RAP)` — instancia o modelo RAP
- `load_nn(checkpoint_name, model)` — carrega os pesos do checkpoint do disco
- `model.to(device) -> model.eval()` — move para GPU/CPU e ativa modo de inferencia
- Em caso de falha: `model = None` , `is_trained = False` — desabilita previsoes

Fase 2 — Construcão de Tensores de Entrada

Tensor	Metodo	Shape Saida	Conteudo
Map	<code>tensor_factory.generate_map_tensor(ticks, map_name, knowledge)</code>	<code>[1, 3, 64, 64]</code>	Posicoes companheiros, inimigos visiveis, utilitarios + bomba
View	<code>tensor_factory.generate_view_tensor(ticks, map_name, knowledge)</code>	<code>[1, 3, 64, 64]</code>	Mascara FOV 90 graus, entidades visiveis, zonas utilitarios
Motion	<code>tensor_factory.generate_motion_tensor(ticks, map_name)</code>	<code>[1, 3, 64, 64]</code>	Trajectoria 32 ticks, campo velocidade, delta mira
Metadata	<code>FeatureExtractor.extract(tick_data, map_name, context)</code>	<code>[1, 1, 25]</code>	Vetor canonico 25-dim (vida, posicao, economia, etc.)

A **ponte PlayerKnowledge** (`_build_knowledge_from_game_state()`) filtra os dados segundo o principio NO-WALLHACK: apenas as informacoes legitimamente disponiveis ao jogador (companheiros, inimigos visiveis, ultimas posicoes conhecidas com decaimento) sao codificadas nos tensores mapa e vista. Se a construcão do conhecimento falha, o sistema degrada ao modo legacy (tensores vazios).

Fase 2b — Modo POV (R4-04-01):

Modo	Condicao	Comportamento
POV Mode	<code>USE_POV_TENSORS=True</code> + <code>game_state</code> fornecido	Constroi <code>PlayerKnowledge</code> do game state -> tensores POV com semantica de canal dedicada
Legacy Mode	<code>USE_POV_TENSORS=False</code> (default)	Tensores standard alinhados com os dados de treinamento

Atencao (R4-04-01): Os tensores POV usam uma semantica dos canais diferente (Ch0=companheiros, Ch1=inimigos ultimos conhecidos) em relacao aos dados de treinamento standard (Ch0=inimigos, Ch1=companheiros). Usar tensores POV com um modelo treinado em modo legacy produzira resultados **nao confiaveis**. O modo POV e valido apenas se o modelo foi treinado com dados POV.

Fase 3 — Inferencia Neural

```
with torch.no_grad():
    out = self.model(view_frame=view_t, map_frame=map_t,
                     motion_diff=motion_t, metadata=meta_t,
                     hidden_state=self._last_hidden) # NN-40: estado persistente
    self._last_hidden = out["hidden_state"] # Mantem para o proximo tick
```

`torch.no_grad()` desabilita o calculo de gradientes (apenas inferencia, nenhum treinamento). O parametro `hidden_state` (NN-40) permite manter o estado recorrente da memoria entre ticks consecutivos, evitando o cold-start a cada avaliacao.

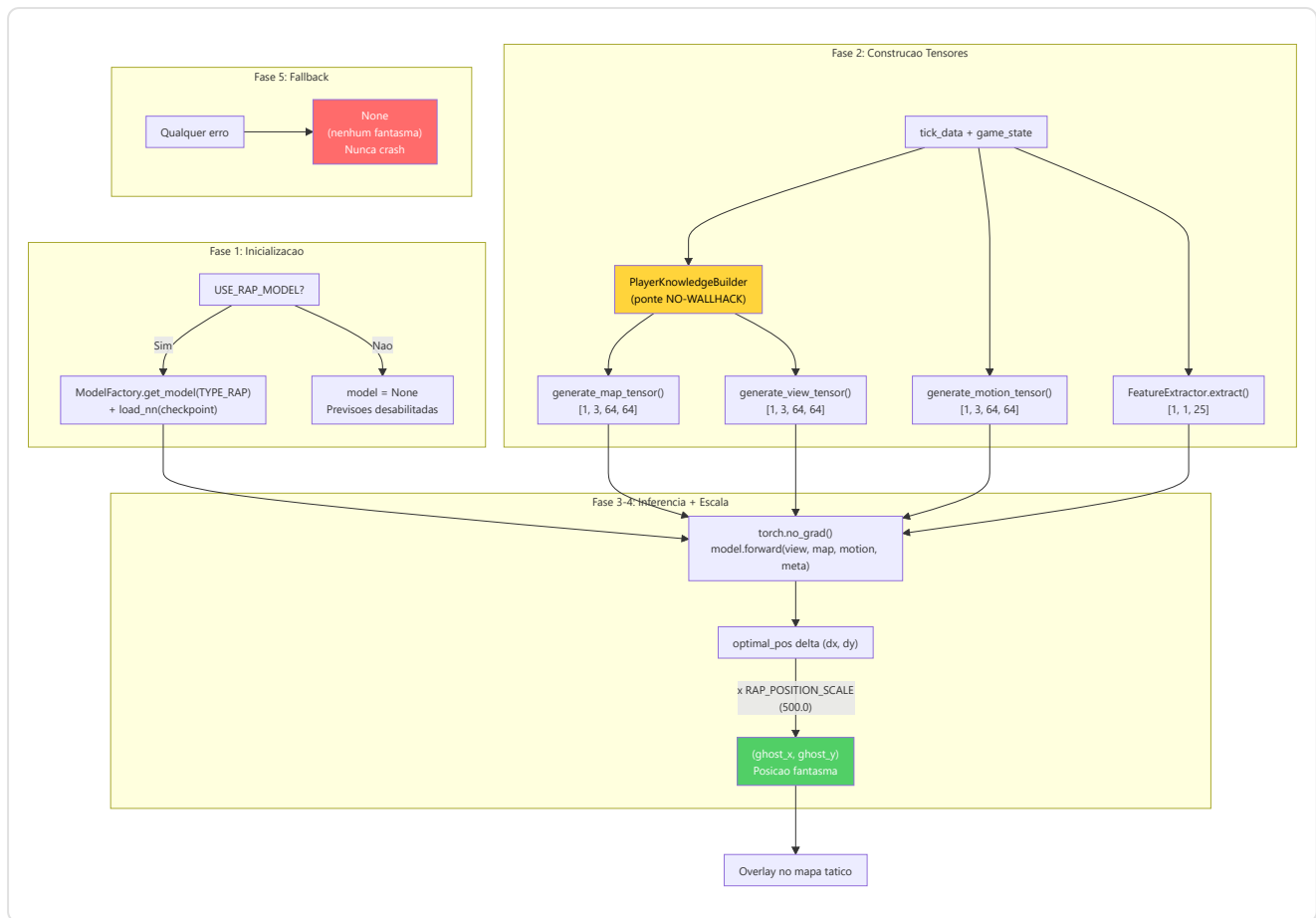
Fase 4 — Decodificacao e Escala de Posicao

```
optimal_delta = out["optimal_pos"].cpu().numpy()[0] # [dx, dy, dz]
ghost_x = current_x + (optimal_delta[0] * RAP_POSITION_SCALE) # x 500.0
ghost_y = current_y + (optimal_delta[1] * RAP_POSITION_SCALE) # x 500.0
return (ghost_x, ghost_y)
```

O modelo produz um delta normalizado em [-1, 1] que e escalado para coordenadas mundo via `RAP_POSITION_SCALE = 500.0` (de `config.py`). A constante e compartilhada entre GhostEngine e overlay para garantir coerencia.

Fase 5 — Fallback Gradual (5 modos)

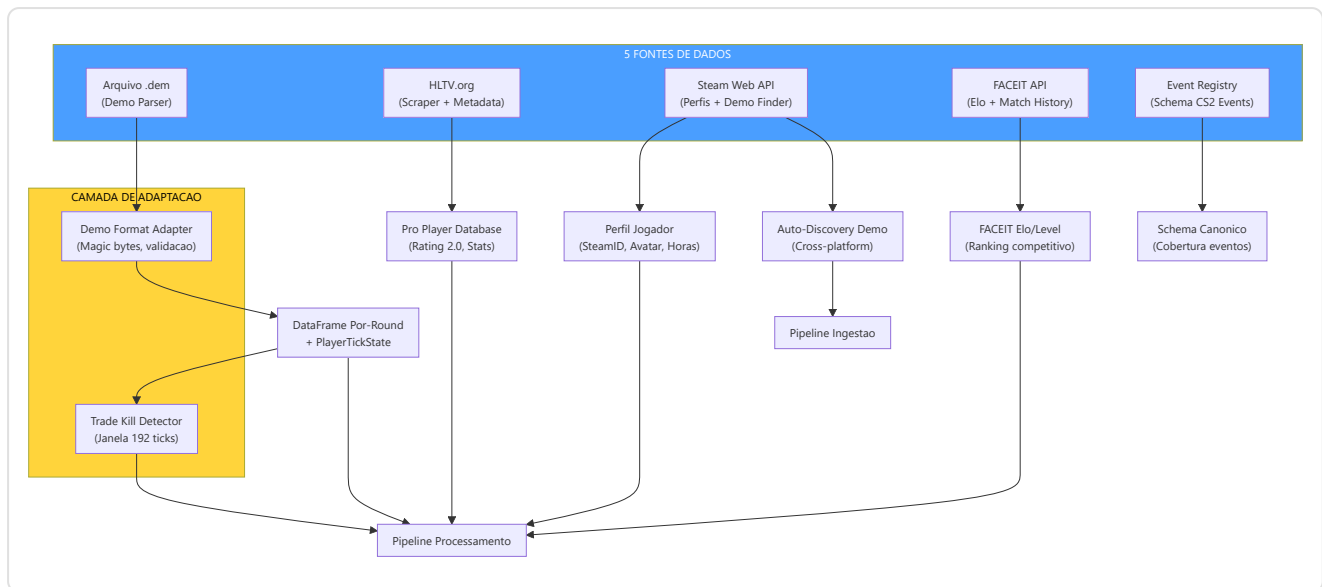
Modo Fallback	Condicao	Comportamento
Modelo desabilitado	<code>USE_RAP_MODEL=False</code> (default)	Skip carregamento, nenhuma previsao (<code>None</code>)
Checkpoint ausente	Treinamento nao concluido	<code>model = None</code> , <code>is_trained = False</code> , previsoes desabilitadas
Nome do mapa ausente	Nenhum contexto espacial	Retorna <code>None</code> imediatamente
Erro PlayerKnowledge	Construcao conhecimento falhou	Degrada para tensores legacy
Erro de inferencia	<code>RuntimeError</code> / <code>CUDA OOM</code>	Log erro, retorna <code>None</code>



5. Subsistema 1B — Fontes de Dados

Pasta no programa: `backend/data_sources/` **Arquivos:** `demo_parser.py`, `demo_format_adapter.py`, `event_registry.py`, `trade_kill_detector.py`, `round_context.py`, `hltv_scraper.py`, `steam_api.py`, `steam_demo_finder.py`, `faceit_api.py`, `faceit_integration.py`, `__init__.py`

O subsistema Fontes de Dados é o **ponto de entrada de todos os dados externos** no sistema. Coleta informações de 5 fontes distintas: arquivos demo CS2, estatísticas HLTV, perfis Steam, dados FACEIT e registry de eventos do jogo.



-Demo Parser (`demo_parser.py`)

Wrapper robusto em torno da biblioteca `demoparser2` para a extracao de estatisticas de arquivos demo CS2. O rating e calculado atraves do **modulo de avaliacao unificado** `feature_engineering/rating.py`, que define as baselines HLTV 2.0:

Constante	Valor	Significado
<code>BASELINE_KPR</code>	0.679	Media pro: kills por round
<code>RATING_BASELINE_SURVIVAL</code>	0.317	Media pro: taxa de sobrevivencia
<code>RATING_BASELINE_KAST</code>	0.70	Media pro: Kill/Assist/Survive/Trade %
<code>RATING_BASELINE_ADR</code>	73.3	Media pro: dano medio por round
<code>RATING_BASELINE_ECON</code>	85.0	Media pro: eficiencia economica

`parse_demo(demo_path, target_player=None)` : Entry point principal. Validacao de existencia do arquivo, parsing de eventos `round_end` para contar os rounds, depois extracao estatistica completa via `_extract_stats_with_full_fields()`. Retorna `pd.DataFrame` vazio em caso de qualquer erro (fail-safe).

`_extract_stats_with_full_fields(parser, total_rounds, target_player)` : Calcula todas as 25 features agregadas obrigatorias para o banco de dados:

- Estatisticas base: `avg_kills`, `avg_deaths`, `avg_adr`, `kd_ratio`
- Variancia: `kill_std`, `adr_std` (via `_compute_per_round_variance`)
- Estatisticas avancadas: `avg_hs`, `accuracy`, `impact_rounds`, `econ_rating`
- Rating HLTV 2.0 aproximado (aproximacao hand-tuned, nao formula oficial)

A guarda de parsing (`parse_guard.py`). `demoparser2` e uma extensao Rust, e uma demo malformada nao levanta uma execucao Python normal: faz o **codigo Rust entrar em panico**, que o pyo3 traduz numa `PanicException`. Aquela classe deriva de `BaseException`, nao de `Exception` - portanto todo `except`

Exception do pipeline a deixava passar, e uma unica demo corrompida interrompia a sessao inteira de ingestao, apesar de os docstrings prometerem o contrario.

A correcao nao podia ser `except pyo3_runtime.PanicException`, porque **o pyo3 cria aquela classe preguiçosamente, no primeiro panico**: antes disso o modulo nem sequer e importavel. A guarda reconhece portanto a execucao **pelo nome da classe**:

```
_PROPAGATE = (KeyboardInterrupt, SystemExit, GeneratorExit)

def is_parse_error(exc: BaseException) → bool:
    if isinstance(exc, _PROPAGATE):
        return False          # estes devem SEMPRE propagar
    if isinstance(exc, Exception):
        return True
    return type(exc).__name__ == "PanicException"
```

O idiomia no ponto de chamada e `except BaseException as exc: if not is_parse_error(exc): raise`. A disciplina e explicita e vale a pena enuncia-la: a unica `BaseException` que este sistema se permite absorver e o panico nomeado; interrupcao de teclado, saida do processo e fechamento de um gerador sempre passam.

O timeout de parsing e real. A versao anterior usava `with ThreadPoolExecutor(...)`: o `FutureTimeoutError` era levantado corretamente, mas na saida do bloco `with` o `shutdown(wait=True)` implicito ficava esperando justamente a thread de parsing travada. O timeout disparava e o chamador ficava bloqueado do mesmo jeito. Hoje `_run_with_parse_timeout()` gerencia o executor a mao e fecha com `shutdown(wait=False, cancel_futures=True)`, **abandonando** a thread orfa e registrando-a em log como erro em vez de esconde-la. O teste `test_parse_timeout_real.py` trava um worker por 30 segundos e exige que o chamador retorne em menos de 5.

-Demo Format Adapter (`demo_format_adapter.py`)

Camada de resiliencia para o tratamento de versoes diferentes do formato demo CS2.

Constantes de validacao:

Constante	Valor	Descricao
<code>DEMO_MAGIC_V2</code>	<code>b"PBDEMS2\x00"</code>	Magic bytes CS2 (Source 2 Protobuf)
<code>DEMO_MAGIC_LEGACY</code>	<code>b"HL2DEMO\x00"</code>	Magic bytes CS:GO legacy (nao suportado)
<code>MIN_DEMO_SIZE</code>	10 x 1024 ² (10 MB)	DS-12: demos CS2 reais sao 50+ MB, arquivos menores sao certamente corrompidos ou incompletos
<code>MAX_DEMO_SIZE</code>	5 x 1024 ³ (5 GB)	Cap de segurancia

Dataclass:

- `FormatVersion(name, magic, description, supported)` — especifica uma versao conhecida do formato

- `ProtoChange(date, description, affected_events, migration_notes)` — registro de uma mudança protobuf conhecida

FORMAT_VERSIONS: Dicionario com dois formatos conhecidos (`cs2_protobuf` suportado, `csgo_legacy` nao suportado).

PROTO_CHANGELOG: Lista cronologica das mudancas conhecidas no formato protobuf CS2 (para resiliencia a futuras atualizacoes).

DemoFormatAdapter.validate_demo(path): Validacao em 3 fases: (1) existencia e tamanho dentro dos bounds, (2) leitura magic bytes para identificacao do formato, (3) verificacao do suporte do formato detectado.

-Event Registry (`event_registry.py`)

Registro canonico de **todos os eventos de jogo CS2** derivado dos dumps SteamDatabase.

GameEventSpec dataclass com 7 campos: `name`, `category` (round/combat/utility/economy/movement/meta), `fields` (dict campo->tipo), `priority` (critical/standard/optional), `implemented` (bool), `handler_path` (opcional), `notes`.

Categorias de eventos registrados:

Categoria	Eventos	Prioridade Critica	Implementados
Round	<code>round_end</code> , <code>round_start</code> , <code>round_freeze_end</code> , <code>round_mvp</code> , <code>begin_new_match</code>	<code>round_end</code>	1/5
Combat	<code>player_death</code> , <code>player_hurt</code> , <code>player_blind</code> , etc.	<code>player_death</code>	parcial
Utility	<code>flashbang_detonate</code> , <code>hegrenade_detonate</code> , <code>smokegrenade_expired</code> , etc.	—	parcial
Economy	<code>item_purchase</code> , <code>bomb_planted</code> , <code>bomb_defused</code> , etc.	<code>bomb_planted/defused</code>	parcial
Movement	<code>player_footstep</code> , <code>player_jump</code> , etc.	—	nao
Meta	<code>player_connect</code> , <code>player_disconnect</code> , etc.	—	nao

Funcoes utilitarias: `get_implemented_events()` -> lista de eventos implementados.

`get_coverage_report()` -> relatorio de cobertura por categoria.

Nota (F6-33): Os `handler_path` nao sao validados em runtime — se os modulos handlers sao movidos, as referencias tornam-se silenciosamente obsoletas. Adicionar validacao `hasattr/callable` ao dispatch dos eventos se a confiabilidade for critica.

-Trade Kill Detector (`trade_kill_detector.py`)

Identifica os **trade kills** — kills de retaliacao dentro de uma janela temporal — das sequencias de morte no demo.

Constante: `TRADE_WINDOW_TICKS = 192` (3 segundos a 64 ticks/s, o tickrate padrao CS2).

`TradeKillResult` dataclass:

- `total_kills`, `trade_kills`, `players_traded`, `trade_details`
- Propriedades calculadas: `trade_kill_ratio`, `was_traded_ratio`

Algoritmo (derivado de cstat-main): Para cada kill K no tick T: olha para tras no tempo procurando kills efetuadas pela vitima. Se a vitima matou um companheiro de equipe do killer de K dentro de

`TRADE_WINDOW_TICKS`, marca K como trade kill e a vitima original como "was traded". **Restricao same-**

round: As kills candidatas ao trade devem pertencer ao **mesmo round** (`round_num` identico). Os trades cross-round nao sao contabilizados — esta e uma distincao tatica importante porque um trade tem significado estrategico apenas dentro do mesmo round, onde influencia diretamente a economia numerica do confronto.

`build_team_roster(parser)`: Constroi mapeamento `player_name → team_num` dos ticks iniciais da partida (usa o 10 percentil dos ticks para estabilidade da atribuicao).

`get_round_boundaries(parser)`: Extrai os ticks de limite entre rounds do evento `round_end`.

-Steam API (`steam_api.py`)

Cliente para a Steam Web API com retry e backoff exponencial.

Constantes: `MAX_RETRIES = 3`, `BACKOFF_DELAYS = [1, 2, 4]` segundos.

`_request_with_retry(url, params, timeout=5)`: Wrapper HTTP GET com 3 tentativas para erros de conexao/timeout. Nao efetua retry em erros HTTP 4xx/5xx (propaga ao chamador).

Funcoes principais:

- `resolve_vanity_url(vanity_url, api_key)` -> resolve um URL personalizado Steam para um SteamID de 64 bits
- `fetch_steam_profile(steam_id, api_key)` -> recupera perfil do jogador (nome, avatar, horas de jogo). Auto-resolve vanity URL se a entrada nao for numerica

-Steam Demo Finder (`steam_demo_finder.py`)

Auto-discovery das demos CS2 da instalacao Steam local.

`SteamDemoFinder` classe com estrategia de deteccao em 3 niveis:

Prioridade	Metodo	Plataforma
1	Registry Windows (<code>winreg</code>)	Windows
2	Caminhos comuns (gerados dinamicamente para cada drive)	Windows/Linux/macOS
3	Variaveis de ambiente	Todas

Deteccao dinamica de drive (Windows): Usa `windll.kernel32.GetLogicalDrives()` para enumerar todos os drives disponiveis, depois busca `Program Files (x86)/Steam`, `Program Files/Steam`, `Steam` em cada drive.

SteamNotFoundError : Excecao especifica quando a instalacao Steam nao pode ser localizada.

Nota (F6-11): A descoberta do caminho Steam e duplicada em `ingestion/steam_locator.py` (primario). Este modulo e suplementar (varre diretorios replay). Consolidacao adiada; assegurar mesma precedencia dos caminhos ao modificar a resolucao.

-Modulo HLTV (`backend/data_sources/hltv/`)

O subsistema HLTV e composto por 3 modulos especializados (`stat_fetcher.py`, `flaresolverr_client.py`, `docker_manager.py`) que colaboram para extrair estatisticas profissionais do site HLTV.org, superando as protecoes anti-scraping do Cloudflare:

HLTVStatFetcher (`stat_fetcher.py`) — orquestrador principal do scraping:

- **Preflight robots.txt** antes de qualquer crawl
- Descobre os jogadores atraves da pagina do ranking dos times (`fetch_top_teams`), depois para cada jogador recupera a pagina overview e as sub-paginas (individual, career, opponents, clutches)
- Salva em `ProPlayer` + `ProPlayerStatCard` os campos: `rating_2_0`, `kpr`, `dpr`, `adr`, `kast`, `impact`, `headshot_pct`, `maps_played`, `opening_kill_ratio`, `opening_duel_win_pct`, `clutch_win_count`, `multikill_round_pct`, `detailed_stats_json`; os times e os rosters em `ProTeam`

Estatisticas extraidas e salvas em `ProPlayerStatCard` :

Categoria	Estatisticas
Core	<code>rating_2_0</code> , <code>kpr</code> (Kill/Round), <code>dpr</code> (Death/Round), <code>adr</code> (Damage/Round)
Eficiencia	<code>kast</code> (Kill/Assist/Survival/Trade %), <code>headshot_pct</code> , <code>impact</code>
Abertura	<code>opening_kill_ratio</code> , <code>opening_duel_win_pct</code>
Tracos (JSON)	Firepower (<code>kpr_win</code> , <code>adr_win</code>), Entrying (<code>traded_deaths_pct</code>), Utility (<code>flash_assists</code>)
Aprofundamentos (JSON)	Clutch (1on1/1on2/1on3), Multikill (3k/4k/5k), Carreira (<code>rating</code> por periodo)

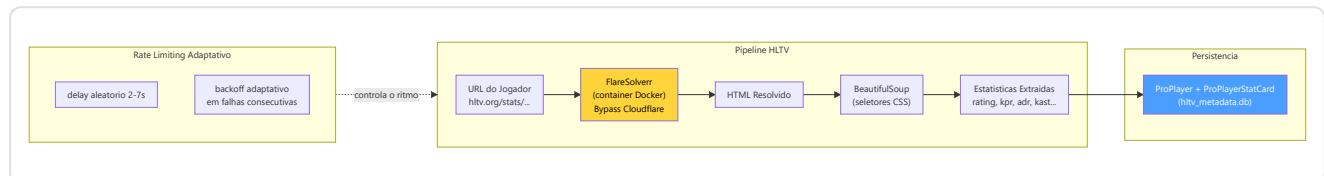
Rate limiting adaptativo (em `stat_fetcher.py`): atraso aleatorio entre requisicoes de `CRAWL_DELAY_MIN_SECONDS = 2` – `CRAWL_DELAY_MAX_SECONDS = 7` segundos, com **backoff adaptativo** que cresce com `_consecutive_failures` — apos falhas consecutivas o fetcher desacelera progressivamente em vez de insistir.

DockerManager (`docker_manager.py`) — gerenciamento do container FlareSolverr com estrategia de inicializacao em cascata:

1. **Fast path:** retorna `True` se ja estiver saudavel
2. **Docker start:** tenta `docker start flaresolverr`

3. **Docker Compose fallback:** tenta `docker compose up -d`
4. **Health polling:** verifica a disponibilidade a cada 3s (`_POLL_INTERVAL_S`) por no maximo 45s (`_MAX_WAIT_S`)

FlareSolvrrClient (`flaresolvrr_client.py`) — bypass automatico dos JavaScript challenges do Cloudflare. Todas as requisicoes HTTP sao roteadas atraves do FlareSolvrr em `http://localhost:8191/v1` (imagem Docker fixada `flaresolvrr:v3.4.6` , unico servico do `docker-compose.yml` do projeto). Retry: `_MAX_RETRIES = 3` com backoff exponencial base 5 (5/15/45s), timeout de 60s por requisicao. O HTML resolvido e passado ao BeautifulSoup para o parsing.



Nota arquitetural: `data_sources/hltv/` e o unico subsistema de scraping HLTV: nao existe (mais) nenhum modulo `HLTVApiService` , `CircuitBreaker` ou `BrowserManager` — a resiliencia esta confiada ao backoff adaptativo do fetcher e aos retries do cliente FlareSolvrr. O daemon de sincronizacao (`hltv_sync_service.py` , processo separado) esta documentado na Parte 3.

Selecao automatica do pro de referencia: O `HybridCoachingEngine` utiliza os dados de `hltv_metadata.db` (tabelas `ProPlayer` , `ProTeam` , `ProPlayerStatCard` mais as tabelas estendidas `ProEvent` , `ProTournament` , `ProHead2Head` , `ProMapRecord`): quando gera uma analise, encontra automaticamente o jogador pro cujo `rating_2_0` esta mais proximo do do usuario e o nomeia nos feedbacks ("seu ADR e inferior ao de [nome do pro]"), via `_find_best_match_pro()` em `coaching_service.py` e `_get_pro_name()` em `hybrid_engine.py` .

`hltv_scraper.py` (entry point em `data_sources/`):

- `run_hltv_sync_cycle(limit=50)` — orquestrador do ciclo de sincronizacao das estatisticas pro

-FACEIT API e Integracao (`faceit_api.py` , `faceit_integration.py`)

faceit_api.py : Funcao unica `fetch_faceit_data(nickname)` que recupera Elo e Level FACEIT para um dado nickname. Requer `FACEIT_API_KEY` da configuracao. Retorna `{faceit_id, faceit_elo, faceit_level}` ou dicionario vazio em caso de erro.

faceit_integration.py : Cliente FACEIT completo com rate limiting:

Parametro	Valor	Descricao
<code>BASE_URL</code>	<code>https://open.faceit.com/data/v4</code>	Endpoint API FACEIT v4
<code>RATE_LIMIT_DELAY</code>	6 segundos	10 req/min = 1 req a cada 6s (tier gratuito)

FACEITIntegration classe com:

- `_rate_limited_request(endpoint, params)` — requisições com rate limiting automático; em HTTP 429 respeita o header `Retry-After` (cap 300s, default 60s) com `MAX_429_RETRIES = 3`
- Gerenciamento de match history, detalhes da partida e download de demos
- Execução dedicada `FACEITAPIError`

-TensorFactory — Fabrica de Tensores (`backend/processing/tensor_factory.py`)

A **TensorFactory** é o **sistema perceptivo** do RAP Coach: converte o estado de jogo bruto em 3 tensores-imagem que o modelo neural pode "ver". Cada tensor é uma imagem de 3 canais que codifica uma dimensão diferente da situação tática: **mapa** (onde todos estão), **vista** (o que o jogador pode ver) e **movimento** (como esta se movendo).

Configurações:

Parametro	<code>TensorConfig</code> (Inferencia)	<code>TrainingTensorConfig</code> (Treinamento)
<code>map_resolution</code>	128 x 128	64 x 64
<code>view_resolution</code>	224 x 224	64 x 64
<code>sigma</code> (blur gaussiano)	3.0	3.0
<code>fov_degrees</code>	90 graus	90 graus
<code>view_distance</code>	2000.0 unidades mundo	2000.0 unidades mundo

Nota (F2-02): `TrainingTensorConfig` reduz a resolução de 128/224 para 64/64, obtendo uma **economia de memória de ~12x**. O contrato `AdaptiveAvgPool2d` na `RAPPerception` produz 128-dim independentemente da resolução de entrada, mas esta garantia é implícita — uma asserção em runtime é recomendada.

Constantes de rasterização:

Constante	Valor	Proposito
OWN_POSITION_INTENSITY	1.5	Brilho marcador posicao propria
ENTITY_TEAMMATE_DIMMING	0.7	Companheiros renderizados mais escuros que inimigos
ENTITY_MIN_INTENSITY	0.2	Intensidade minima entidade visivel
ENEMY_MIN_INTENSITY	0.3	Intensidade minima inimigo visivel
BOMB_MARKER_RADIUS	50.0	Raio circulo bomba (unidades mundo)
BOMB_MARKER_INTENSITY	0.8	Opacidade circulo bomba
TRAJECTORY_WINDOW	32 ticks	Janela trajetoria (~0.5s a 64 Hz)
VELOCITY_FALLOFF_RADIUS	20.0	Celulas grid para gradiente radial velocidade
MAX_SPEED_UNITS_PER_TICK	4.0	Velocidade maxima CS2 (64 ticks/s)
MAX_YAW_DELTA_DEG	45.0	Limiar flick para deteccao de mira

Os 3 Rasterizadores:

1. Rasterizador Mapa — `generate_map_tensor(ticks, map_name, knowledge)` -> `Tensor(3, res, res)`

Canal	Modo Player-POV (com PlayerKnowledge)	Modo Legacy (sem knowledge)
Ch0	Companheiros (sempre conhecidos) + posicao propria (intensidade 1.5)	Posicoes inimigos
Ch1	Inimigos visiveis (intensidade total) + ultimos inimigos conhecidos (decaimento exponencial)	Posicoes companheiros
Ch2	Zonas utilitarios (fumaca/molotov) + overlay bomba	Posicao jogador

2. Rasterizador Vista — `generate_view_tensor(ticks, map_name, knowledge)` -> `Tensor(3, res, res)`

Canal	Modo Player-POV	Modo Legacy
Ch0	Mascara FOV (cone geometrico 90 graus da direcao de olhar)	Mascara FOV
Ch1	Entidades visiveis: companheiros (dimmed x0.7) + inimigos visiveis (intensidade ponderada por distancia)	Zona perigo (areas NAO cobertas por FOV acumulado, capped em 8 ticks)
Ch2	Zonas utilitarios ativos (circulos fumaca/molotov em unidades mundo)	Zona segura (recentemente visivel mas nao em FOV corrente)

3. Codificador Movimento — `generate_motion_tensor(ticks, map_name)` -> `Tensor(3, res, res)`

Canal	Conteudo
Ch0	Trajectoria ultimos 32 ticks — intensidade proporcional a recencia (mais novo = 1.0, mais antigo -> 0)
Ch1	Campo velocidade — gradiente radial do jogador, modulado pela velocidade corrente [0, 1]
Ch2	Movimento de mira — magnitude delta yaw como blob gaussiano na posicao do jogador

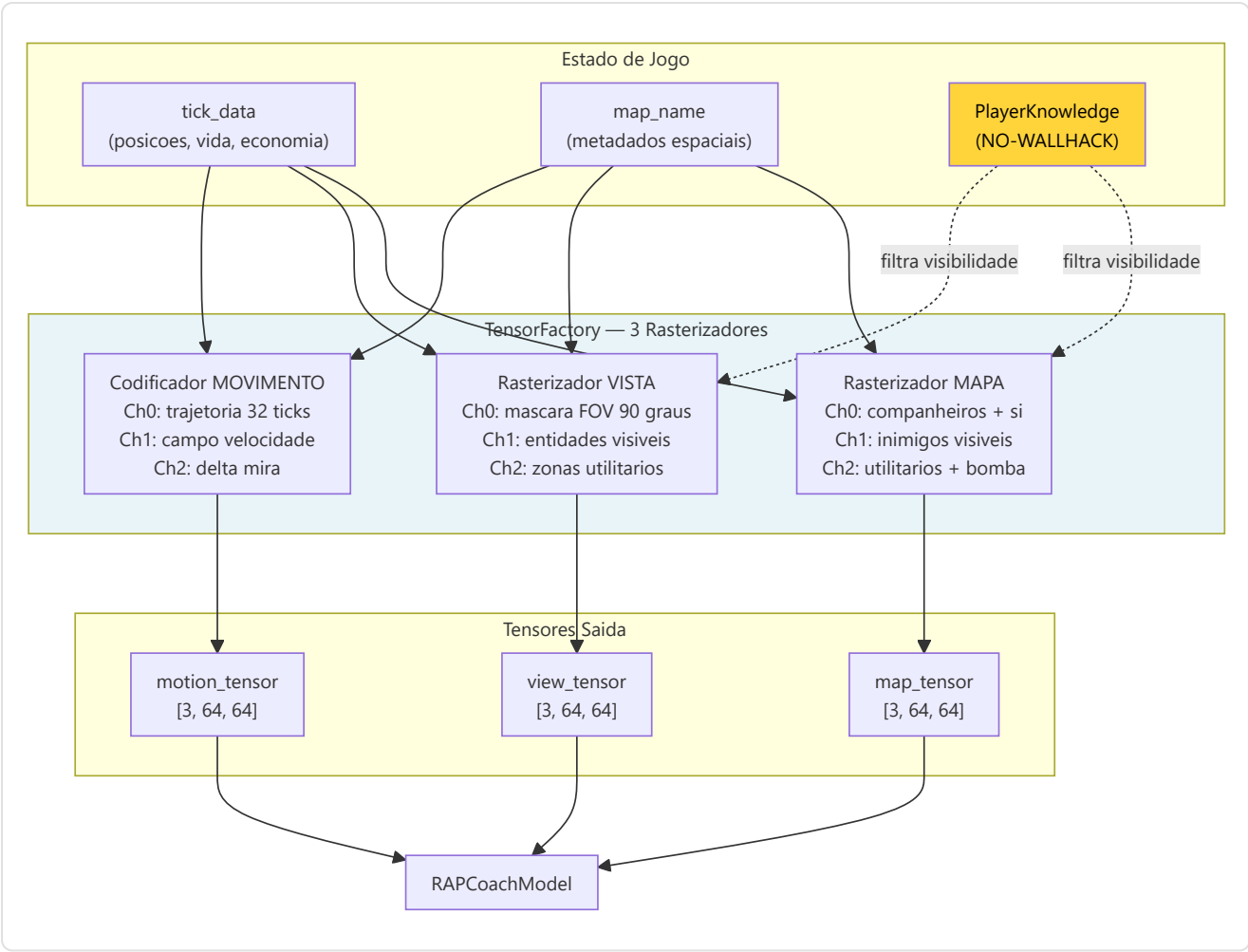
Nota (F2-03): Demos a 128 ticks/s comprimem a velocidade na metade inferior do intervalo [0, 1]; normalizacao consciente do tick-rate aguardando implementacao.

Integracao NO-WALLHACK: Quando `PlayerKnowledge` e fornecida, os rasterizadores mapa e vista codificam **apenas o estado visivel ao jogador**. As posicoes inimigas do ultimo avistamento decaem exponencialmente no tempo. As zonas utilitarios sao visiveis apenas se no FOV ou conhecidas pelo radar. Quando `knowledge=None`, o sistema degrada ao modo legacy para retrocompatibilidade.

Metodos helper:

- `_world_to_grid(x, y, meta, resolution)` — Conversao coordenadas mundo -> grid. **Nota C-03:** Unico Y-flip (`meta.pos_y - y`) para evitar dupla inversao
- `_normalize(arr)` — Normalizacao para [0, 1]. **Nota M-10:** `arr / max(max_val, 1.0)` para prevenir amplificacao do ruido em canais esparsos
- `_generate_fov_mask(player_x, player_y, yaw, meta, resolution)` — Mascara conica 90 graus da direcao de olhar, limitada por distancia (aproximacao 2D top-down)

Acesso Singleton: `get_tensor_factory()` — double-checked locking, thread-safe.



-Índice Vetorial FAISS (backend/knowledge/vector_index.py)

O **VectorIndexManager** fornece busca semantica de alta velocidade para o sistema de conhecimento RAG (Retrieval-Augmented Generation) do coach. Usa FAISS (Facebook AI Similarity Search) com **IndexFlatIP** sobre vetores L2-normalizados, obtendo efetivamente uma **busca por similaridade cosseno** em tempo sublinear.

Índices Duais:

Índice	Fonte DB	Conteúdo
"knowledge"	Tabela TacticalKnowledge	Embedding conhecimento tático (estrategias, posicoes, utilitarios)
"experience"	Tabela CoachingExperience	Embedding experiencias de coaching (feedback, correcoes, conselhos)

Tipo de índice: **faiss.IndexFlatIP** (Inner Product) sobre vetores L2-normalizados. Como $\cos(a, b) = \frac{a \cdot b}{||a|| \times ||b||}$, normalizando os vetores a norma unitaria, o produto interno **equivale exatamente** a similaridade cosseno. Intervalo resultante: [0, 1] onde 1 = identico.

API publica:

Metodo	Descricao
<code>search(index_name, query_vec, k)</code>	Busca os k vetores mais similares. Lazy rebuild se dirty. Retorna <code>List[(db_id, similarity)]</code>
<code>rebuild_from_db(index_name)</code>	Reconstrucao completa do indice da tabela DB. Thread-safe. Retorna contagem de vetores
<code>mark_dirty(index_name)</code>	Marca o indice para reconstrucao lazy (no proximo <code>search()</code>)
<code>index_size(index_name)</code>	Retorna <code>index.ntotal</code> ou 0 se nao construido

Persistencia em disco:

- Formato: `{persist_dir}/{index_name}.faiss` + `{index_name}_ids.npy`
- Salvamento: `faiss.write_index()` + `np.save()`
- Carregamento: automatico em `__init__` via `faiss.read_index()` + `np.load()`
- Diretorio default: `~/cs2analyzer/indexes/`

Thread-safety: Um unico `threading.Lock()` protege todas as operacoes de leitura/escrita sobre os indices, os flags dirty e as operacoes de rebuild. FAISS `IndexFlatIP` e thread-safe para leituras concorrentes.

Reconstrucao lazy (`mark_dirty()`): Quando novos dados sao inseridos nas tabelas Knowledge ou Experience, o indice e marcado como "dirty" em vez de reconstruido imediatamente. A reconstrucao ocorre apenas no proximo `search()`, evitando rebuilds multiplos durante insercoes batch.

Normalizacao vetorial:

```
norms = ||embedding||_2 por linha
normalized = embedding / max(norms, 1e-8)    # estabilidade numerica
IndexFlatIP.add(normalized)
```

Fallback gradual: Se `faiss-cpu` nao esta instalado, o singleton `get_vector_index_manager()` retorna `None` e o sistema degrada automaticamente para busca brute-force (mais lenta mas funcionalmente equivalente). Isto permite ao programa funcionar mesmo em sistemas onde FAISS nao esta disponivel.

Over-fetching com constantes explicitas: Para gerenciar cenarios de pos-filtragem (category, map_name, confidence, outcome), a busca recupera mais resultados que o necessario: `k x OVERFETCH_KNOWLEDGE = k x 10` para a Knowledge Base (filtro por categoria + mapa), `k x OVERFETCH_EXPERIENCE = k x 20` para o Experience Bank (filtro por mapa + confidence + outcome + scoring composito). O multiplicador 20x para as experiencias e o dobro em relacao ao conhecimento porque os filtros sao mais restritivos (4 criterios vs 2), portanto serve um pool inicial mais amplo para garantir resultados suficientes apos a filtragem.

-Contexto dos Rounds (`round_context.py`)

O modulo **Round Context** e a **grade temporal** do sistema de ingestao: converte os ticks brutos dos arquivos demo em coordenadas significativas "round N, tempo T segundos" que cada outro modulo pode usar para contextualizar os eventos de jogo.

Funcoes publicas:

Funcao	Entrada	Saida	Complexidade
<code>extract_round_context(demo_path)</code>	Caminho arquivo <code>.dem</code>	DataFrame: <code>round_number</code> , <code>round_start_tick</code> , <code>round_end_tick</code>	O(n) parsing de eventos
<code>extract_bomb_events(demo_path)</code>	Caminho arquivo <code>.dem</code>	DataFrame: <code>tick</code> , <code>event_type</code> (planted/defused/exploded)	O(n) parsing de eventos
<code>assign_round_to_ticks(df_ticks, round_context, tick_rate)</code>	DataFrame ticks + limites rounds	DataFrame enriquecido com <code>round_number</code> , <code>time_in_round</code>	O(n log m) via <code>merge_asof</code>

Construcao dos limites de round (`extract_round_context`):

O modulo analisa dois tipos de eventos do arquivo demo:

- `round_freeze_end` — o tick em que termina o freeze time e comeca a acao (os jogadores podem se mover)
- `round_end` — o tick em que o round termina (vitoria/derrota)

Para cada round, combina o ultimo `round_freeze_end` que precede o `round_end` correspondente.

Fallback: se nao e encontrado um evento `round_freeze_end` para um dado round (possivel em demos corrompidos ou partidas interrompidas), usa o `round_end` do round anterior como inicio, registrando um warning no log.

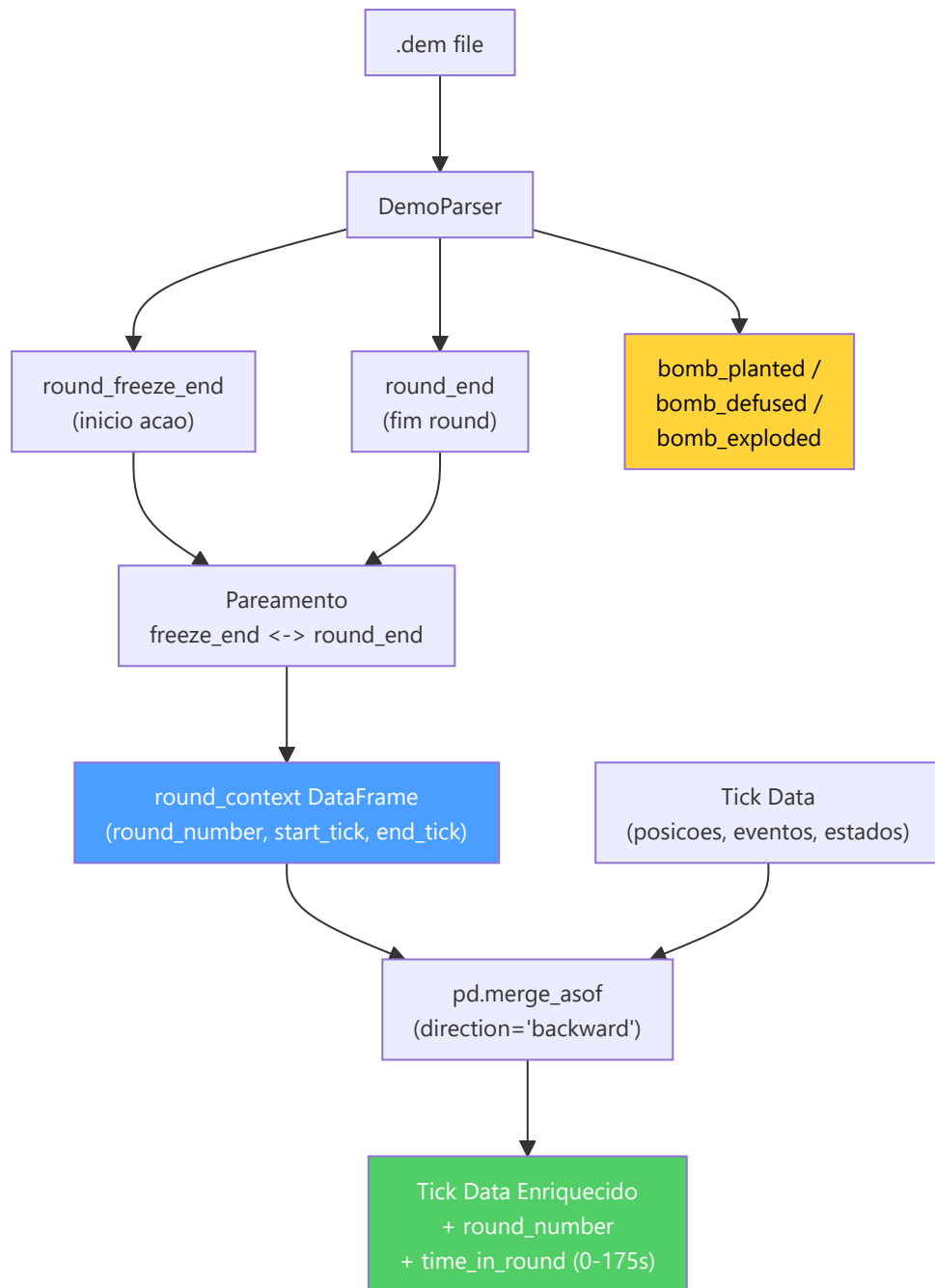
Extracao de eventos de bomba (`extract_bomb_events`):

Extrai tres tipos de eventos: `bomb_planted`, `bomb_defused` e `bomb_explored`. A adicao de `bomb_explored` (remediacao H-07) permite distinguir entre rounds ganhos por explosao e rounds ganhos por eliminacao, uma informacao critica para a analise tatica pos-plant.

Atribuicao de round aos ticks (`assign_round_to_ticks`):

Usa `pd.merge_asof` com `direction="backward"` para uma atribuicao eficiente O(n log m): para cada tick, encontra o ultimo `round_start_tick ≤ tick`. Calcula `time_in_round = (tick - round_start_tick) / tick_rate`, limitado a [0.0, 175.0] segundos (duracao maxima de um round CS2). Os ticks antes do primeiro round (warmup) sao atribuidos ao round 1.

Nota: O uso de `merge_asof` no lugar de um loop Python transforma uma operacao O(n x m) em O(n log m), fundamental para demos com milhoes de ticks e 30+ rounds.

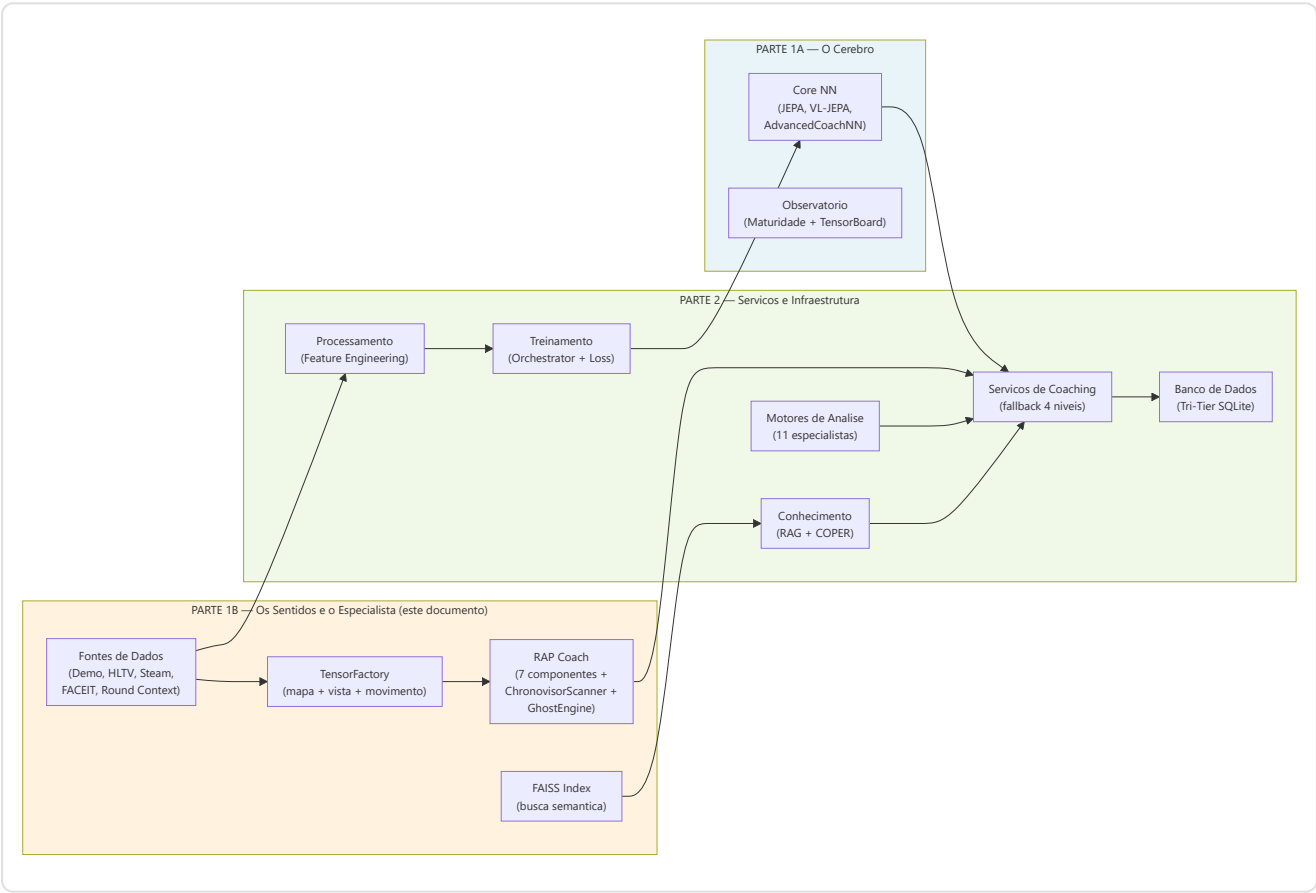


Gerenciamento de erros: Cada fase de parsing é protegida por try/except com logging estruturado. Se o parsing falha completamente ou não são encontrados eventos `round_end`, a função retorna um DataFrame vazio — os módulos a jusante (ex: `RoundStatsBuilder`) devem gerenciar este caso gracefully.

Resumo da Parte 1B — Os Sentidos e o Especialista

A Parte 1B documentou os **dois pilares perceptivos e diagnósticos** do sistema de coaching:

Subsistema	Papel	Componentes Chave
2. RAP Coach	O medico especialista — arquitetura de 7 componentes para coaching completo em condicoes POMDP	Percepcao (ResNet de 3 fluxos, 24 conv), Memoria (LTC 512 unidades NCP + Hopfield 4 cabecas + atraso de ativacao NN-MEM-01 + RAPMemoryLite fallback LSTM), Estrategia (4 especialistas MoE + SuperpositionLayer), Pedagogia (Value Critic + Skill Adapter), Atribuicao Causal (5 categorias, sinal utilitarios aprendido), Posicionamento (Linear 256->3), Comunicacao (template), ChronovisorScanner (3 escalas temporais + 50K ticks safety limit + deduplicacao cross-escala + ScanResult estruturado), GhostEngine (pipeline 4-tensores com POV mode R4-04-01, hidden_state NN-40, fallback em 5 niveis)
1B. Fontes de Dados	Os sentidos — adquirem e estruturam dados do mundo externo	Demo Parser (demoparser2 + rating HLTV 2.0 unificado), Demo Format Adapter (magic bytes PBDEMS2), Event Registry (schema CS2), Trade Kill Detector (janela 3s ciente do tick), Steam API (retry + backoff), Steam Demo Finder (cross-platform), HLTV (FlareSolvrr v3.4.6 + delay adaptativo 2-7s + preflight robots.txt → hltv_metadata.db, 7 tabelas Pro*), FACEIT API (10 req/min, Retry-After em 429), TensorFactory (3 rasterizadores NO-WALLHACK), FAISS (IndexFlatIP sobre embeddings 384-dim), Round Context (merge_asof O(n log m))



Continua na Parte 2 — *Serviços de Coaching, Coaching Engines, Conhecimento e Recuperação, Motores de Análise (11), Processamento e Feature Engineering, Módulo de Controle, Progresso e Tendências, Banco de Dados e Storage (Tri-Tier), Pipeline de Treinamento e Orquestração, Funções de Perda*

